
title: Bugproof Pipelines

description: This module tackles real-world data challenges with the Oxford Flowers dataset, showing how poor pipelines can break even the best models. You'll learn to build custom Datasets, implement transform pipelines, split data correctly, and apply ...

Decoding a Secret Message

- The PyTorch Workflow
 - Moving beyond fundamental model building to a more comprehensive workflow

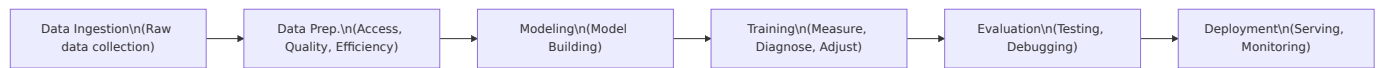
Character-Level Decoding Approach

- The strategy for reading the handwritten letter involves two main steps:
 - Segmenting the letter into individual character images (e.g., 'x', 'z', 'a', 'n')
 - Building a model to classify each individual character
- Once each character is classified, the goal is to reconstruct the entire message from the sequence of predicted characters
- **[Result]** The initial model attempt failed to produce a correct reconstruction, resulting in illegible text:

```
Tcq Zsoqcdgc,  
Brlc fnc luHrqgn groqzc mz ermde xcpp.  
Tr drf wrqecf fr iccl fnc psaz mdfcqzfmde sdy cdesemde. Ksuac uro gropy nstc fnc  
zfoycdfz fqu fr ycgryc bu bezzu nsdyxqmfmde. Hnsf bmenf ac s amf frr gnspccidemde  
fnroen.  
P sb mblqczzy uro sqc sapc fr qcsy fnmz.  
-Fdyqcx
```

Working with Images and the ML Pipeline

- Images represent a significant increase in data complexity
 - Each image contains hundreds or thousands of individual pixel values
- The focus will be on mastering the complete machine learning pipeline
 - This includes moving beyond basic model building to advanced topics
 - Key components to master include loss functions and optimizers
 - Managing hardware resources, such as GPUs, becomes essential



MNIST Database

- A dataset of handwritten digits used for practicing handwriting recognition
- **[Challenge]** Neural networks must learn to account for significant human variation, such as:
 - Neatness vs. messiness
 - Slanting of characters

MNIST Database Characteristics

- **[Why use it?]** Because it is simple and highly structured, making it easy to train a neural network quickly
 - Models can often be trained in minutes rather than hours
- Key technical specifications:
 - Dimensions: 28 by 28 pixels
 - Color: Grayscale
 - Alignment: Neatly centered
 - Labels: 10 digits (0 to 9)

Future Deep Dives into the ML Pipeline

- The upcoming material will provide a more granular look at the machine learning workflow using PyTorch:
 - **Data Management:** Focusing on tools to effectively load and manage large, messy, real-world datasets
 - **Advanced Modeling and Training:** Moving beyond sequential models to build more flexible architectures and gaining a deeper understanding of how training works

Moving Toward Convolutional Neural Networks

- MNIST serves as the stepping stone for building an initial image classifier
- The next architectural goal is to implement a Convolutional Neural Network (CNN)
 - CNNs are specialized neural network architectures designed to process grid-like data, such as images

Universal Data Handling Patterns

- The core tools and logic used to manage simple data (e.g., delivery records) are the same as those used for complex data (e.g., millions of images)
- **[Why this matters]** Once you master the patterns for simple data, handling complex image data becomes much more intuitive
- PyTorch provides a consistent set of data handling tools that scale across different data types

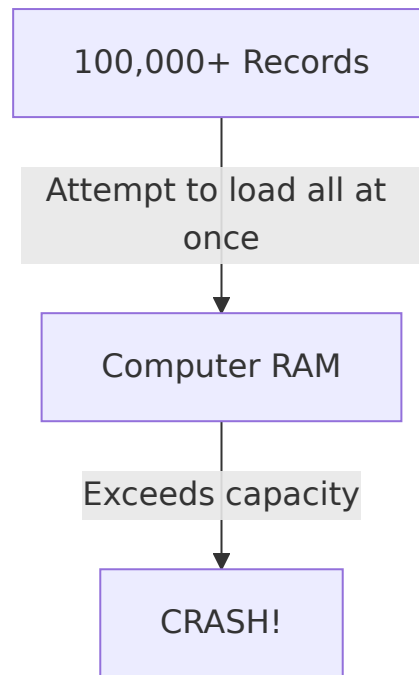
Data Ingestion

Scaling Challenges and Memory Constraints

- As datasets grow from small collections (e.g., 10 records) to massive scales (e.g., 100,000 or millions of records), loading everything at once becomes problematic
- **[The Problem]** Attempting to load all records into memory simultaneously can exhaust available RAM
 - This is especially true when adding auxiliary data like weather, traffic, or driver information to the primary dataset
 - Running out of memory can cause the entire system to crash

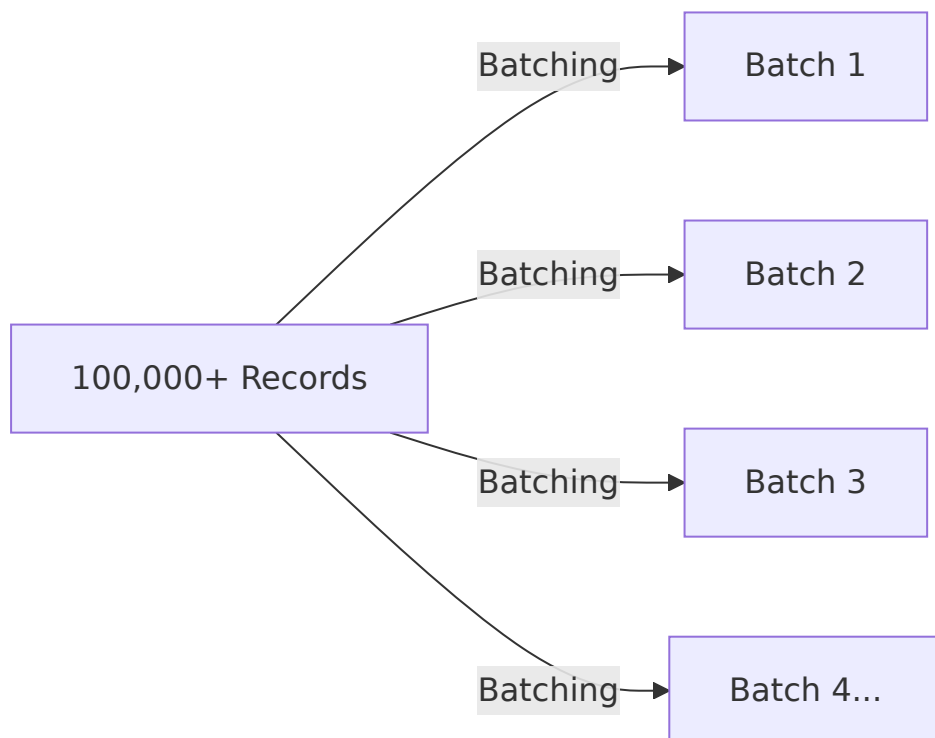
```
# Try to load all delivery data into memory
all_distances = []
all_times = []

for i in range(100000):
    distance, time = load_delivery_record(i)
    all_distances.append(distance)
    all_times.append(time)
```



Batching as a Solution

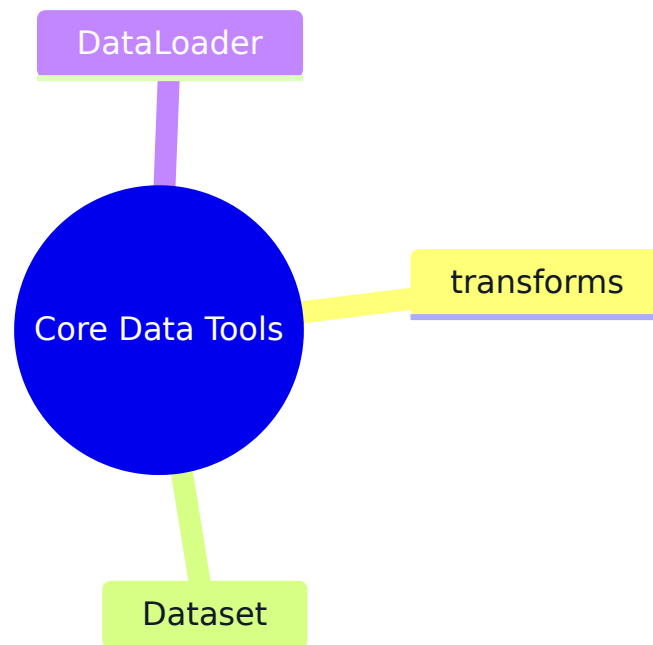
- To avoid exhausting RAM when dealing with massive datasets, data should be loaded piece by piece
- **[The Method]** Use batches, which are smaller, manageable chunks of the full dataset



Core PyTorch Data Tools

- Three essential utilities designed to work together to process, format, and serve data to a model:

- **Transforms:** Handles data preprocessing (e.g., resizing, format conversion)
- **Dataset:** Manages how data is accessed
- **DataLoader:** Handles the batching and serving of the data



Implementing Transforms in PyTorch

- Transforms are operations that run on each individual data point as it is being loaded
- **[Purpose]** To prepare data for the model before it enters the training process
- `transforms.Compose` is used to chain multiple operations together in a specific order

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((mean,), (std,))
])
```

Common Transform Operations

- `transforms.ToTensor()` : Converts data into a PyTorch tensor
- `transforms.Normalize((mean,), (std,))` : Adjusts the data values
 - **[Why use it?]** Neural networks train much more effectively when inputs are small numbers, ideally centered around zero

Detailed Transform Mechanics

- `transforms.ToTensor()` :

- Converts data into PyTorch tensors
- Scales values to fall between 0 and 1
- `transforms.Normalize((mean,), (std,))`:
 - Adjusts values even further by centering them around 0
 - Scales the distribution using standard deviation
 - **[Why use it?]** This helps the model train much more effectively

The Dataset Object

- A utility used to wrap data for efficient access
- **[How it works]** It fetches each individual sample from the disk only when it is specifically asked for
 - This prevents the need to preload the entire dataset into memory at once
- Encapsulates the logic required to manage data within the ML pipeline
- **[Core Responsibilities]**
 - Tracking where data is stored on the disk
 - Defining how to load a specific sample
 - Determining the total number of samples available
 - Applying specified transforms to each sample during the loading process

Initializing Pre-built Datasets

- PyTorch provides many pre-built datasets that can be initialized with specific arguments

```
dataset = SomeDataset('./data', train=True, download=True,
transform=transform)
```

- **Key Arguments:**
 - `train`: Used to choose between training and test/validation sets
 - `download`: If set to `True`, it automatically downloads the dataset if it is not already available locally
 - `transform`: Specifies the preprocessing steps to be applied to each sample

Interacting with Datasets and DataLoaders

- **[Accessing Data]** Individual samples can be retrieved from a dataset using standard Python indexing

- Example: `first_item = dataset[0]`
- **[The DataLoader's Role]** Once a dataset is defined, the `DataLoader` is used to serve that data in manageable batches
 - This is a critical part of the pipeline that enables training on massive datasets by requesting only one batch at a time
- **[DataLoader Configuration]**
 - `batch_size` : Determines how many individual samples are served in a single batch
 - `shuffle` : Determines whether the order of the data is randomized for each epoch

```
dataset_loader = DataLoader(dataset, batch_size=32, shuffle=True)
```

The PyTorch Data Pipeline

- A repeatable pattern for preparing and serving data for training
- **[Core Principle]** Load only what you need, when you need it
 - This allows the pipeline to handle massive datasets (thousands or millions of records) without overwhelming computer memory

Pipeline Steps

```
# 1. Define your transforms
transform = SomeTransform() # e.g., ToTensor + Normalize

# 2. Create Datasets with transforms
train_dataset = YourDataset('./data', train=True, transform=transform)

# 3. Create DataLoaders
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)

# 4. Training loop
for batch_idx, (data, labels) in enumerate(train_loader):

    # Data arrives in batches, already transformed

    # Your model processes this batch
    output = model(data)
```



Overview of the ML Pipeline with PyTorch

The PyTorch Workflow

- The machine learning pipeline follows a specific progression beyond data management:
 - **Data Ingestion:** Raw data collection
 - **Data Prep:** Access, quality, and efficiency
 - **Modeling:** Model building
 - **Training:** Measure, diagnose, and adjust
 - **Evaluation:** Testing and debugging
 - **Deployment:** Serving and monitoring

Model Building Patterns

- While `nn.Sequential` is a common and effective pattern for simple models, PyTorch also supports creating models by defining custom classes.
- **[Comparison of Patterns]**

- `nn.Sequential` : Useful for quick, linear stacks of layers
- Class-based (`nn.Module`): Provides more flexibility for complex architectures

```
# Sequential pattern example
model = nn.Sequential(
    nn.Linear(1, 20),
    nn.ReLU(),
    nn.Linear(20, 1)
)
```

```
# Class-based pattern example
class ExampleModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.layer_1 = nn.Linear(1, 20)
        self.relu = nn.ReLU()
        self.layer_2 = nn.Linear(20, 1)

    def forward(self, x):
        x = self.layer_1(x)
        x = self.relu(x)
        x = self.layer_2(x)
        return x
```

Anatomy of an `nn.Module` Class

When using the class-based pattern, every PyTorch module is structured into two distinct functional parts:

- `__init__` (The Setup Phase):
 - This is where you define the layers and components that will make up your model.
 - **[Analogy]** It is like gathering your tools and laying them out on a workbench before starting a job.
- `forward` (The Execution Phase):

- This method describes the actual path the data takes through the layers defined in `__init__`.
- This is where the logic of the model's architecture is implemented, determining how input is transformed into output.

Executing the Forward Pass

- **[The Correct Way]** Instead of calling the `forward` method explicitly, you should call the model instance directly as if it were a function
 - Correct: `output = model(data)`
 - Incorrect/Avoid: `output = model.forward(data)`
- **[Why this works]** `nn.Sequential` and custom classes inheriting from `nn.Module` are designed so that calling the object triggers the necessary internal PyTorch logic to execute the `forward` method

```
# Correct way to run the model
model = ExampleModel()
output = model(data)

# This is equivalent to how nn.Sequential works internally
model = nn.Sequential(nn.Linear(1, 20), nn.ReLU(), nn.Linear(20, 1))
output = model(data)
```

The Importance of Calling the Model Instance

- **[Why not `model.forward(data)` ?]** Calling the object directly performs more than just the forward pass
 - It triggers essential internal bookkeeping
 - It makes necessary internal checks
 - It tracks the math required for gradient calculation
 - It sets up the model for future updates

The Necessity of `super().__init__()`

- When defining a custom class inheriting from `nn.Module`, you must include `super().__init__()` in the `__init__` method

- **[What happens if you skip it?]** Omitting this call will cause an `AttributeError` because PyTorch cannot properly initialize the underlying module structure

```
class ExampleModel(nn.Module):  
    def __init__(self):  
        super().__init__() # Essential to avoid AttributeError  
        self.fc1 = nn.Linear(784, 128)  
        ...
```

The Role of `super().__init__()` in Parameter Tracking

- **[Why it is essential]** It initializes the underlying `nn.Module` structure so PyTorch can set up a system to track all learnable parameters
 - These parameters include the weights and biases that are updated during training
 - Without this call, PyTorch has no way to register the layers you define (e.g., `self.fc1`)
- **[Consequence of omission]** Attempting to create a model without it results in an `AttributeError` because the module cannot assign parameters before the base class is initialized

The Training Phase

- The training phase is the stage in the ML pipeline where the model is adjusted based on performance



- **The Standard Training Loop Sequence**
 - This core sequence is used in almost all PyTorch training implementations:

```
for epoch in range(epochs):  
    optimizer.zero_grad()           # 1. Clear out old calculations  
    outputs = model(X)              # 2. Forward pass  
    loss = loss_function(outputs, y) # 3. Calculate error  
    loss.backward()                  # 4. Calculate improvements  
    (gradients)  
    optimizer.step()                 # 5. Update weights
```

The Importance of Training Loop Order

- The standard sequence in a training loop must be followed strictly to ensure the model learns correctly
- **[The Risk of Silent Failure]** If the order is incorrect, the code may still execute without throwing errors, but the model will not learn properly

Common Sequencing Errors

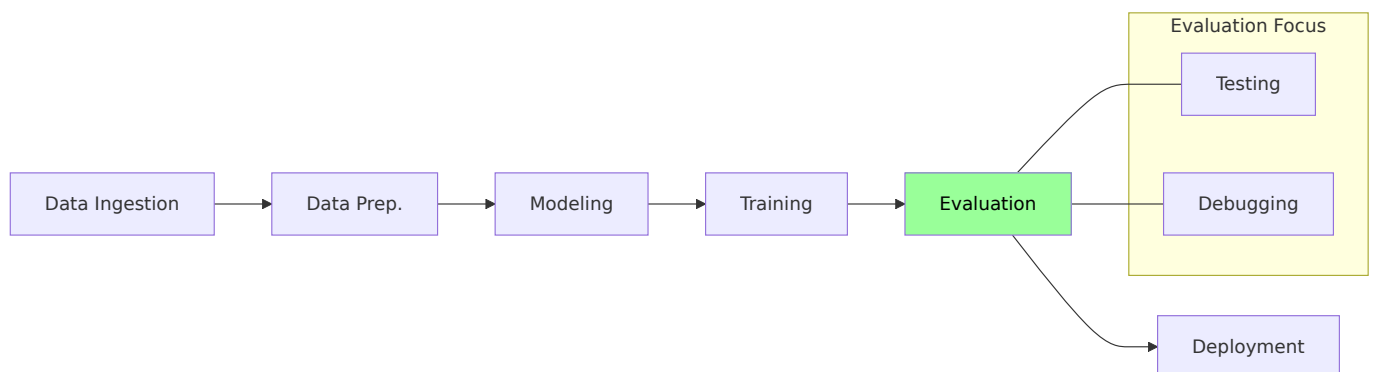
- **Swapping `loss.backward()` and `optimizer.step()`**
 - If `optimizer.step()` is called before `loss.backward()`, the model updates its weights based on the gradients from the *previous* batch rather than the current one
- **Placing `optimizer.zero_grad()` after `loss.backward()`**
 - Doing this effectively wipes out the gradients calculated by the backward pass before the optimizer can use them to update the weights
- **Placing `optimizer.zero_grad()` outside the epoch loop**
 - This causes gradients from every batch to accumulate (pile up) indefinitely, rather than being cleared for each new calculation

Evaluation

- The stage where you test your model on new data that it hasn't previously seen and wasn't trained on
- **[Purpose]** To determine if the model has actually learned patterns that generalize to new inputs

Evaluation Pipeline Details

- The evaluation process involves specific tasks within the broader pipeline:



- **Implementation Essentials**

- `model.eval()` : A critical command used to set the model into evaluation mode
- **[Note]** This is distinct from the training mode used during the training phase

Evaluation Mode and Gradient Management

- `model.eval()` : Sets the model into evaluation mode
 - **[Clarification]** This does *not* evaluate the model's performance; it simply switches the mode of certain layers
 - Certain layers (like Dropout or Batch Normalization) behave differently during training versus evaluation
 - Using this mode is also more computationally efficient
- `torch.no_grad()` : A context manager used to disable gradient tracking
 - **[Why use it?]** During evaluation, we don't need to calculate gradients for weight updates
 - If not used, PyTorch continues to store all the intermediate details needed for backpropagation, which wastes memory and processing power

```
model.eval()                # Set evaluation mode
with torch.no_grad():        # Disable gradient tracking
    correct = 0
    total = 0
    for images, labels in test_loader:
        outputs = model(images)
        _, predicted = torch.max(outputs, 1) # Get the class with
highest score
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total
    print(f'Accuracy: {accuracy}%')
```

Measuring Performance via Accuracy

- **[The Goal]** To see how well the model performs on new data that it hasn't seen before
 - Testing on the training set is ineffective because the model might simply memorize the answers rather than learning to apply patterns
- For image classification, performance is often measured using accuracy

- **Accuracy Calculation Logic**

- Count the number of correct predictions
- Divide by the total number of samples
- Multiply by 100 to get a percentage

```
# Example accuracy calculation logic
correct = 0
total = 0
for images, labels in test_loader:
    outputs = model(images)
    _, predicted = torch.max(outputs, 1) # Get the class with highest
score
    total += labels.size(0)
    correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total
print(f'Accuracy: {accuracy}%')
```

Accuracy Example

- Accuracy is the ratio of correct classifications to total attempts
 - For example, if a model correctly predicts 9,500 digits out of 10,000, the accuracy is 95%

Returning to Training Mode

- **[Crucial Step]** If you are returning the model to the training phase after evaluation, you must explicitly switch it back
- `model.train()` : Sets the model back to training mode so that layers like Dropout and Batch Normal Normalization function correctly for weight updates

Summary of the ML Pipeline

- The complete workflow involves several distinct stages:



- The training process follows a repetitive three-step sequence: **Measure, Diagnose, and Update**
- This sequence is implemented through three core lines of code:

```
loss = loss_function(outputs, targets) # -> measure
loss.backward()                       # -> diagnose
optimizer.step()                      # -> update
```

The Three-Step Cycle

- **Measure**
 - Calculated via `loss = loss_function(outputs, targets)`
 - This step determines how far off the model's predictions are from the actual targets
- **Diagnose**
 - Performed via `loss.backward()`
 - This handles the complex math behind the scenes to understand the error
- **Update**
 - Executed via `optimizer.step()`
 - This step actually adjusts the model's parameters based on the diagnosis

Measuring Loss

- The focus of this module is the first step: measuring the loss
- An example of a loss function is `nn.MSELoss()`

```
loss_function = nn.MSELoss()
```

Loss Functions

- Every loss function performs the same fundamental task:
 - It compares the model's predictions to the true answers
 - It produces a single number representing the error
 - **[Key Principle]** The higher the resulting number, the more incorrect the model's predictions are
- **Choosing the right loss function depends on the task type**

Mean Squared Error (MSE)

- Used for regression tasks (e.g., predicting continuous values like delivery times)
- Implemented in PyTorch as:

```
loss_function = nn.MSELoss()
```

Cross Entropy Loss

- Used for classification tasks (e.g., identifying which category an item belongs to)
- Implemented in PyTorch as:

```
loss_function = nn.CrossEntropyLoss()
```

Understanding Mean Squared Error Intuition

- **Error Calculation**
 - An error is the distance between a model's prediction and the actual reality
 - It is calculated by subtracting the target (y_i) from the prediction ($\tilde{y}_i$)
- **Total Loss**
 - The total loss is the average of all these individual errors across the entire dataset
 - **[Analogy]** Think of the loss as a "report card" for how well the model is performing

Real (y_i)	Predicted ($\tilde{y}_i$)	Difference ($y_i - \tilde{y}_i$)
4	6	-2
5	3	+2

The Necessity of Squaring Errors in MSE

- **The Problem with Raw Averages**
 - If you simply average the raw differences (errors), positive and negative errors can cancel each other out
 - **[Example]** If one error is +2 and another is -2, the average error is 0
 - This creates a false impression of perfect performance even when the model is actually making mistakes
- **The MSE Solution**

- We square the differences $(y_i - \tilde{y}_i)^2$ to ensure all errors are positive
- This prevents errors from canceling each other out and ensures every mistake contributes to the final loss score

Real (y_i)	Predicted ($\tilde{y}_i$)	Difference ($y_i - \tilde{y}_i$)	Difference ² ($(y_i - \tilde{y}_i)^2$)
4	6	-2	4
5	3	+2	4
Average		0	4

The Impact of Squaring in MSE

- **[Key Benefits]** MSE serves two critical purposes in a model's training:
 - It ensures all mistakes count (by making them positive)
 - It punishes larger errors more than smaller ones
- **[The Magnitude Effect]** Because errors are squared, a larger mistake becomes exponentially more significant than a small one
 - **[Example]** A 10-minute difference is $10^2 = 100$, whereas a 1-minute difference is $1^2 = 1$
 - This means the 10-minute error is treated as being 100x worse than the 1-minute error
- **Best Use Cases**
 - MSE is ideal for regression tasks where you are predicting continuous values
 - Examples: distances, temperatures, or prices

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \tilde{y}_i)^2$$

- **Implementation in PyTorch**

```
loss_function = nn.MSELoss()
loss = loss_function(predictions, targets)
```

- **[Usage]** Used for classification tasks (e.g., identifying digits in MNIST)
- **[Mechanism]** Instead of picking a single answer, the model outputs a confidence score (probability) for every possible class
 - In MNIST, the model outputs 10 numbers (one for each digit 0-9)

- These scores represent the likelihood of the input belonging to that specific class
- All confidence scores for a single input should sum to 1

[Example Output]

For an image of the digit "3", the model might output the following confidence distribution:

Digit	0	1	2	3	4	5	6	7	8	9
Conf	0.05	0.02	0.08	0.70	0.03	0.04	0.02	0.03	0.02	0.01

- In this case, the model is 70% confident the digit is a 3

Implementation in PyTorch

```
loss_function = nn.CrossEntropyLoss()  
loss = loss_function(outputs, labels)
```

The Intuition of Cross Entropy Loss

- **[The Penalty Mechanism]** Cross Entropy Loss is specifically designed to punish overconfident wrong answers
 - If a model is highly confident in a wrong prediction, the loss will be very high
 - If a model is less confident in a wrong prediction, the loss will be smaller
- **[Goal of Training]** The objective is to train the model to be:
 - Confident about the correct answers
 - Unsure about the incorrect ones

[Example of Overconfidence]

Imagine an image that is actually a digit "3", but the model predicts it is a "7":

Scenario	Confidence in "7"	Loss Magnitude
Overconfident	0.95	Very High
Less Confident	0.55	Smaller

- In the first case, the model is making a massive error by being nearly certain of the wrong class, resulting in a significant loss penalty.

Avoiding Loss Function Mismatches

- **[The Risk of Mixing]** Using the incorrect loss function for a specific task leads to poor training outcomes
 - **MSE for Classification:** While it might technically "work," it is inefficient and can lead to slow or unstable training
 - **Cross Entropy Loss for Regression:** This will likely break the model because it expects probability distributions (values summing to 1) rather than continuous numerical values
- **[Comparing Loss Values]** You cannot compare the raw numerical outputs of different loss functions to determine which model is "better"
 - **[Reasoning]** Different loss functions operate on different scales and mathematical principles
 - **[Example]** An MSE of 0.08 is not inherently "better" than a Cross Entropy Loss of 2.3; they are measuring fundamentally different things

Selecting the Right Loss Function

- **[Decision Rule]** The choice of loss function depends entirely on the nature of the value you are trying to predict
 - **Use MSE (Mean Squared Error)** when predicting a continuous number
 - Examples: temperature, price, or distance
 - **Use Cross Entropy Loss** when predicting a discrete category
 - Examples: a digit, an animal, or a word
- **[Training Goal]** Regardless of the function used, the mathematical objective during training is to minimize the loss value over time

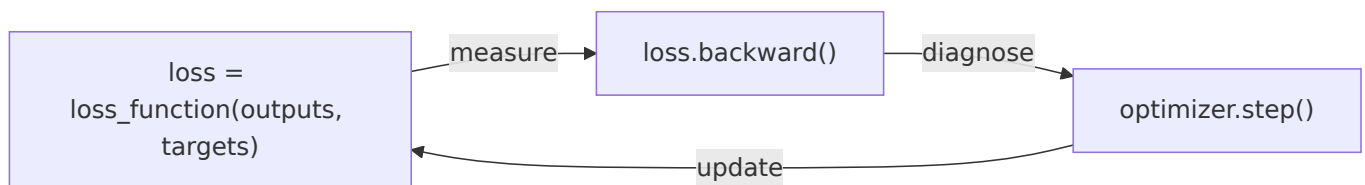
Common Loss Functions and Their Uses

Loss Function	Primary Use Case
<code>nn.MSELoss()</code>	Measures average squared difference; common for regression (predicting numbers)
<code>nn.CrossEntropyLoss()</code>	For multi-class classification (picking 1 out of many categories)
<code>nn.L1Loss()</code>	Measures average absolute difference; regression that is less sensitive to outliers

<code>nn.BCEWithLogitsLoss()</code>	For binary classification (yes/no, true/false decisions)
<code>nn.NLLLoss()</code>	Multi-class classification when using LogSoftmax activation
<code>nn.SmoothL1Loss()</code>	Regression that balances MSE and L1Loss benefits (also called Huber loss)
<code>nn.KLDivLoss()</code>	Measures difference between two probability distributions

The Role of Loss in the Training Loop

Loss functions serve as the "measure" component of the training sequence. The broader cycle involves measuring error, diagnosing the cause, and then updating the model to improve performance.



- **[The Sequence]**
 - **Measure:** Use a loss function to quantify the difference between predictions and targets
 - **Diagnose:** Use the backward pass (`loss.backward()`) to calculate gradients
 - **Update:** Use an optimizer (`optimizer.step()`) to adjust model weights based on those gradients

Optimizers and Gradients

After measuring error with a loss function, the training process moves into diagnosis and improvement.

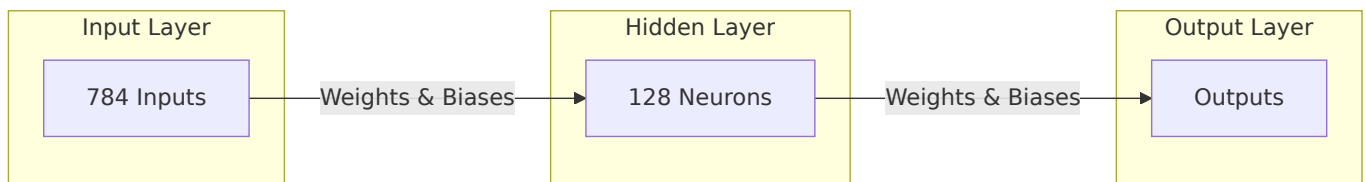
The Training Loop Components

- `loss.backward()`
 - Acts as the "diagnose" step
 - Figures out which specific weights in the model caused the errors in prediction
- `optimizer.step()`
 - Acts as the "update" step

- Uses the information from the diagnosis to adjust the model weights and improve performance

The Mechanics of Diagnosis

- **loss.backward()**
 - Investigates the cause of the measured loss
 - Examines how each weight and bias in the network contributed to the error
- **Neural Network Parameter Complexity**
 - Even small networks involve massive numbers of parameters
 - Each neuron performs a calculation involving:
 - Inputs
 - Weights (multiplied by inputs)
 - Biases (added to the sum)
 - An activation function
 - For example, a network with 784 inputs and 128 hidden neurons involves a significant number of individual weights and biases to track



Parameter Scale and Responsibility

- **Trainable Parameters**
 - Even simple architectures involve a huge number of individual components to track
 - Example network (784 inputs → 128 hidden → 10 outputs):
 - 784
 $times 128 = 100,352$ weights
 - 128 biases (for the hidden layer)
 - 128
 $times 10 = 1,280$ weights
 - 10 biases (for the output layer)
 - **Total: 101,770 trainable parameters**
- **The Detective Work of backward()**

- Acts like a detective investigating a crime
- It examines every single weight and bias to answer: "How much did you contribute to the loss?"
- This is based on the fundamental neuron calculation:

$$z = W_1 \cdot x_1 + W_2 \cdot x_2 + W_3 \cdot x_3 + b$$

- It determines the gradient (contribution) for each specific W and b so the optimizer knows what to adjust.

Gradients

- Diagnostic scores that identify how much and in what direction each weight contributed to the loss
- **The meaning of gradient values:**
 - **Positive values** (e.g., +2.5):
 - Increasing this weight will make the loss worse
 - To reduce loss, the weight should be decreased
 - **Negative values** (e.g., -2.5):
 - Increasing this weight would have helped reduce the loss
 - **Magnitude (Influence):**
 - **Large values** (e.g., +10 or -8.5): Very influential on the loss
 - **Small values** (e.g., +.001 or -.0005): Barely matter to the overall error
- **[Common Misconception]** `loss.backward()` does **not** update the weights
 - It only calculates the gradients (the diagnosis)
 - The actual adjustment of weights is handled by a different step (the optimizer)

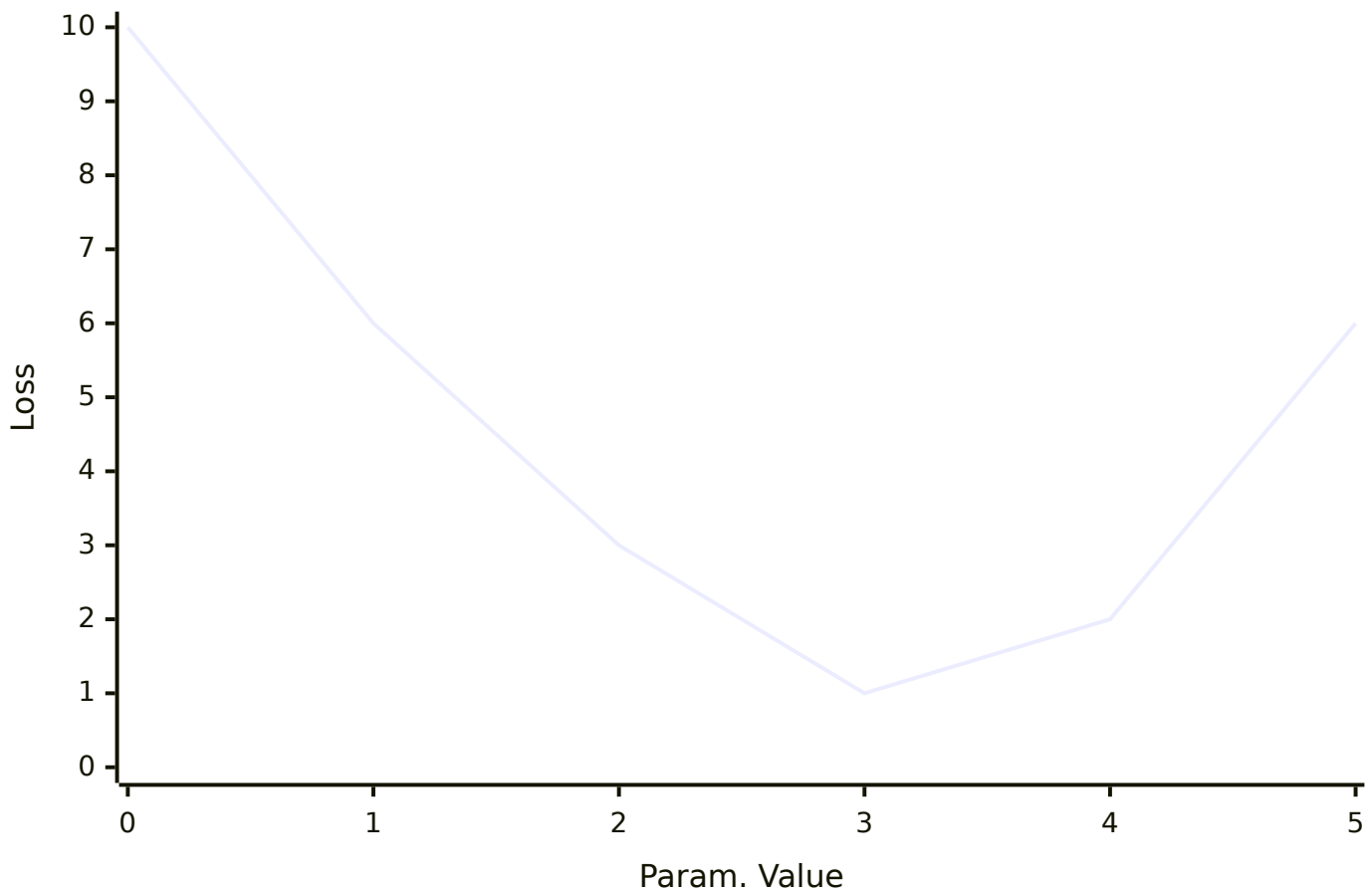
The Goal of Training: Minimizing Loss

- The objective is to minimize the loss function
- **[Analogy]** Minimizing loss is like standing on a hillside and trying to reach the bottom of a valley
 - The gradient tells you the slope at your current position, guiding you toward the minimum

- **The Training Sequence Recap**

- `loss = loss_function(outputs, targets)` → **Measure**
- `loss.backward()` → **Diagnose** (Calculates gradients; does not update weights)
- `optimizer.step()` → **Update** (Adjusts the parameters to reduce the loss)

Loss Landscape



- In this landscape, the goal is to find the point of **Min. Loss**
- The gradient (e.g., $\frac{dy}{dx} = -4$) provides the direction and steepness needed to move toward that minimum.

Gradient Descent and SGD

- The goal is to move "downhill" toward the minimum loss
- **Stochastic Gradient Descent (SGD)** is a straightforward strategy for this movement:
 - **If the gradient is negative:** Increase the weight
 - **If the gradient is positive:** Decrease the weight
 - **[Directional Rule]** Always move in the opposite direction of the gradient to reach lower loss
 - **[Magnitude Rule]:** The size of the adjustment is proportional to the gradient

- Large gradient → Big change
- Small gradient → Small change

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

- The `lr` (learning rate) parameter controls the scale of these updates

The Role of the Learning Rate

- The optimizer does not subtract the gradient directly from the weights
- Instead, it scales the gradient by the learning rate (`lr`) before applying the update
 - **[Example]** If the gradient $\frac{dy}{dx} = 0.5$ and the learning rate is 0.01, the actual step taken is 0.5×0.01
- **[Impact of different learning rates]**
 - **Tiny learning rate:** The steps are so small that it will take an impractical amount of time to reach the minimum
 - **Huge learning rate:** The steps are too large, causing the optimizer to "bounce" back and forth across the valley without settling
 - **Good learning rate:** Provides steady, efficient progress toward the minimum loss

The Adam Optimizer

- An adaptive optimizer that serves as a smarter alternative to SGD
 - It can adjust updates for each weight individually
 - It acts like an "assistant" that knows which weights need large adjustments versus fine-tuning
- **[Why use it?]** It is a popular first-choice because it is reliable, flexible, and often faster than other alternatives
- **[Warning]** Do not reuse the same learning rate from SGD when switching to Adam
 - Using an SGD-sized learning rate with Adam can cause the loss to explode


```
# Adam typically uses a smaller learning rate
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Comparison with SGD
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

Other PyTorch Optimizers

- While SGD and Adam are the most common for most projects, PyTorch offers many other specialized optimizers
 - Examples include `RMSprop`, `AdaGrad`, `Adadelta`, `AdamW`, and `LBFGS`

The Training Loop and `zero_grad()`

- A standard training loop follows a specific 5-step sequence to ensure correct updates

```
# Standard Training Loop Pattern
for epoch in range(500):

    # 1. Reset the optimizer
    optimizer.zero_grad()

    # 2. Make predictions
    outputs = model(inputs)

    # 3. Calculate the loss
    loss = loss_function(outputs, times)

    # 4. Calculate adjustments
    loss.backward()

    # 5. Update the model
    optimizer.step()
```

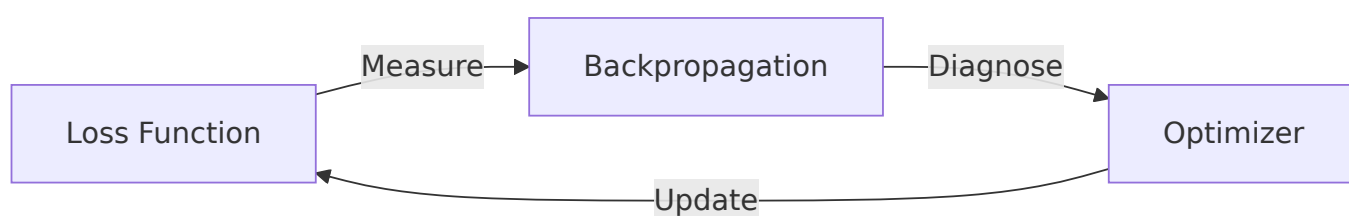
- **[The Importance of Step 1]:** You must call `optimizer.zero_grad()` at the start of every iteration
 - **[Why?]** Because every time you call `loss.backward()`, PyTorch **adds** the newly calculated gradients to whatever is already stored in the parameter buffers
 - **[The Risk]** If you do not reset them, you aren't just diagnosing the current parameters; you are accumulating gradients from all previous steps, which will lead to incorrect weight updates

The Utility of Gradient Accumulation

- PyTorch's default behavior of accumulating gradients is an intentional design choice rather than a mistake
 - **[Why?]** It enables advanced training techniques, such as gradient accumulation or implementing specific custom training schedules
- While useful for these specialized scenarios, most standard projects require manual management via `optimizer.zero_grad()` to prevent incorrect, additive updates during a typical training loop

The Training Loop Conceptual Framework

- The training process can be understood as a three-part cycle of measurement, diagnosis, and adjustment



- **[Loss Function]:** Measures how far the model's predictions are from the target values
- **[Backpropagation]:** Diagnoses how much each individual parameter contributed to the error
- **[Optimizer]:** Uses those diagnostic scores to perform the actual weight updates

Device Management

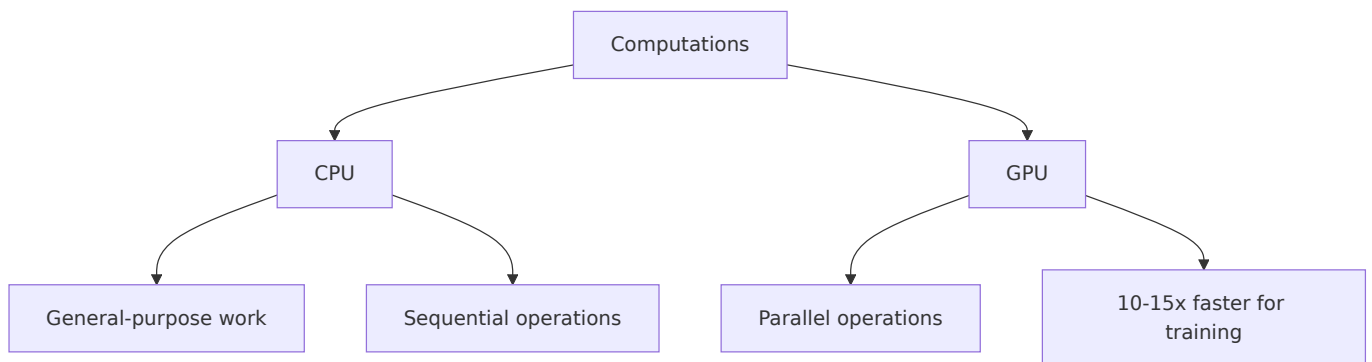
- The next step in building a classifier involves managing hardware devices
 - **[Goal]:** Moving computations to GPUs to enable significantly faster training speeds

The Requirement for Device Consistency

- Every tensor and model resides on a specific device
 - Common devices include the CPU, GPU, or other hardware accelerators
- **[The Critical Rule]**: PyTorch will not automatically move objects between devices
 - If a model is on one device and a tensor is on another, the code will fail
- **[Common Error]**: Mismatched devices trigger a `RuntimeError`
 - Example: `RuntimeError: Expected all tensors to be on the same device`

CPU vs. GPU

- **CPU (Central Processing Unit)**
 - The default device PyTorch uses unless otherwise specified
 - Designed for general-purpose work
 - Executes operations sequentially
- **GPU (Graphics Processing Unit)**
 - An accelerator used to run tensor operations much faster
 - Designed for parallel operations
 - **[Performance]** Can be 10 to 15 times faster than a CPU for training tasks



Selecting a Hardware Device

- Because GPUs are significantly faster for training, it is best practice to check for their availability automatically
- **[Detection]**: Use `torch.cuda.is_available()` to determine if a GPU is accessible to PyTorch
- **[Common Pattern]**: A standard way to define the `device` variable for use throughout a script:

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

CUDA and Device Transfer

- **[The CUDA Keyword]:** Refers specifically to Nvidia GPUs utilizing the CUDA toolkit
 - While other options like MPS (for Apple Silicon) exist, CUDA is the most common standard used in many deep learning environments
- **[Moving to Device]:** Once the `device` variable is defined, the model must be explicitly moved to that hardware to utilize its acceleration

```
model = MyModel().to(device)
```

Managing Devices in the Training Loop

- **[Model Initialization]:** When creating a model, move its parameters to the selected device immediately

```
model = MyModel().to(device)
```

- **[Data Ingestion]:** Within the training loop, each batch of data must also be moved to the same device to avoid `RuntimeErrors`

```
for inputs, targets in dataloader:  
    inputs = inputs.to(device)  
    targets = targets.to(device)
```

- **[Debugging Device Mismatches]:** If you encounter device errors, you can verify where objects are located:
 - **For Tensors:** Use the `.device` attribute

```
print(inputs.device)
```

```
- **For Models**: Models themselves aren't on devices, but their parameters are. Check the device of a single parameter to verify the model's location
```

```
print(next(model.parameters()).device)
```

- **[Troubleshooting Tip]**: When debugging, always double-check that both the model's output and the target labels are on the same device

Common `.to(device)` Pitfall

- **[Non-In-Place Operation]**: Calling `.to(device)` on a tensor does not change the original tensor in place; instead, it creates and returns a new tensor on the specified device
- **[Requirement]**: You must always reassign the result to a variable to ensure the tensor is actually moved

```
# Incorrect: x remains on the original device  
x.to(device)
```

```
# Correct: x is reassigned to the new tensor on the target device  
x = x.to(device)
```

The Training Loop Structure

- A complete training loop involves several coordinated steps, beginning with hardware setup and model/data preparation

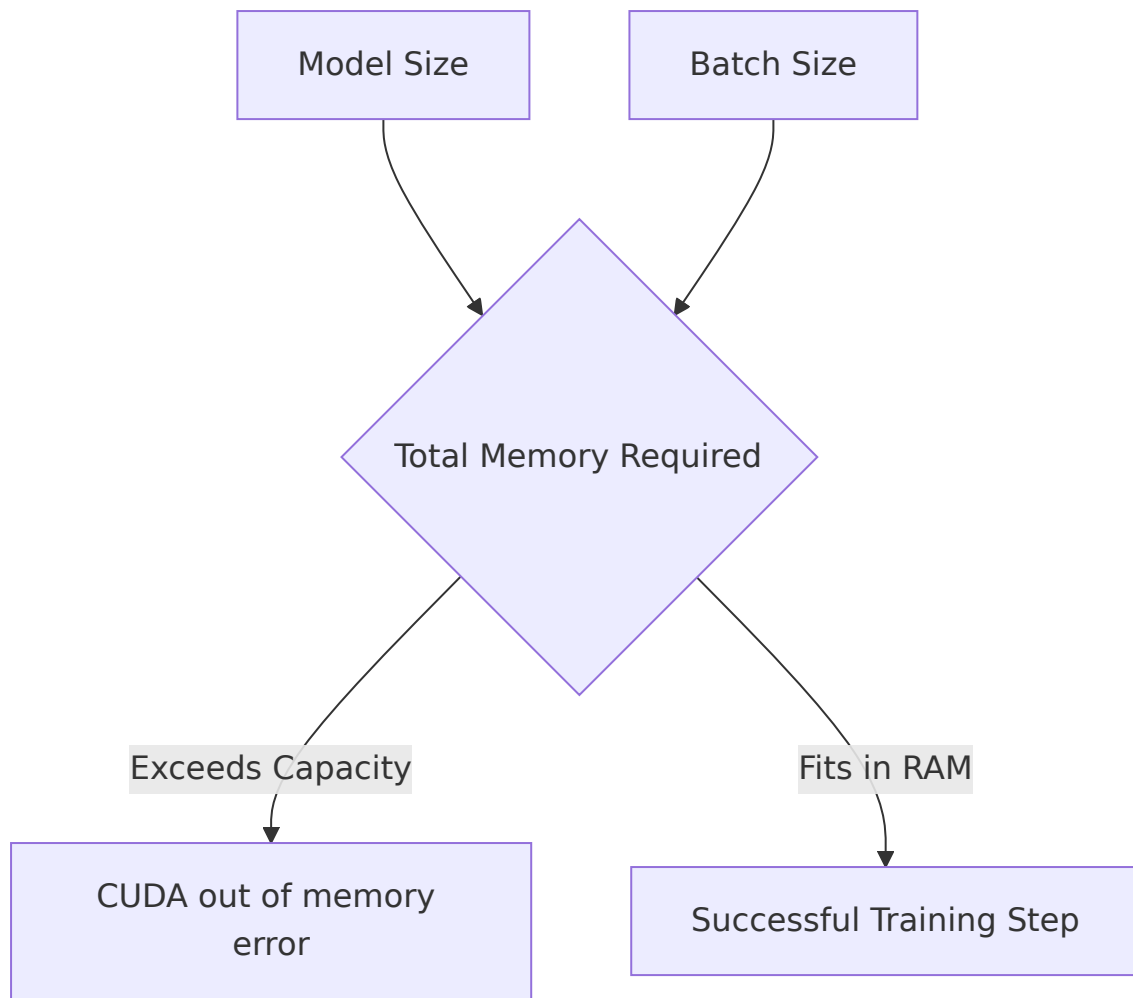
```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = MyModel().to(device)
optimizer = optim.Adam(model.parameters())
loss_function = nn.CrossEntropyLoss()

for inputs, targets in dataloader:
    inputs = inputs.to(device)
    targets = targets.to(device)

    optimizer.zero_grad()
    outputs = model(inputs)
    loss = loss_function(outputs, targets)
    loss.backward()
    optimizer.step()
```

GPU Memory Constraints

- **[The Problem]:** GPU memory (VRAM) is a finite resource. If the combined memory footprint of the model and the current batch exceeds the available GPU RAM, the program will crash
- **[The Error]:** This failure typically manifests as a `CUDA out of memory error`
- **[The Trade-off]:** Batch size is a critical lever in managing this memory:
 - **Small batches:** Easier on memory but make training slower
 - **Large batches:** Faster training but increase the risk of exceeding memory limits



Managing GPU Memory

- **[The Batch Size/Memory Tradeoff]:** The size of each batch directly impacts how much GPU RAM is consumed
 - Larger batches require more memory to store the data and the intermediate activations during the forward and backward passes
 - If the batch is too large for the available hardware, it will trigger a **CUDA out of memory error**
- **[Practical Recommendations]**
 - **Starting Point:** For many systems, a batch size between 32 and 64 is a good baseline
 - **Hardware Dependency:** The ideal size depends on your specific hardware and the complexity of your model architecture
- **[Troubleshooting Memory Errors]**
 - **Primary Fix:** If you encounter a memory error, the most common and effective first step is to **lower your batch size**

Image Classification with MNIST

- **[The Dataset]:** MNIST consists of handwritten digits used for training and testing image classifiers
 - **Composition:** 60,000 training images and 10,000 test images
 - **Dimensions:** Each image is 28 *times* 28 pixels
 - **Format:** Grayscale images

Data Pipeline

Import Libraries

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
```

Data Preprocessing

To prepare image data, `torchvision` provides tools to apply a sequence of transformations using `transforms.Compose`.

```
# Data preprocessing
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])
```

- **`transforms.ToTensor()`**
 - Converts images into PyTorch tensors
 - Scales pixel values from the original range of 0 to 255 down to a range of 0 to 1
- **`transforms.Normalize((mean,), (std,))`**
 - Shifts and scales the tensor values

- Centers the data around 0 using the provided mean and standard deviation

Normalization Context

- **[Why normalize?]** Using the mean (0.1307) and standard deviation (0.3081) of the entire training set makes the data more consistent
 - Consistency helps the model learn faster

Loading the MNIST Dataset

Using `torchvision.datasets.MNIST` allows for easy loading and configuration of the dataset.

```
# Loading the training dataset
train_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=True,
    download=True,
    transform=transform
)

# Loading the test dataset
test_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=False,
    download=True,
    transform=transform
)
```

- **`root='./data'`**
 - Specifies the directory where the dataset files will be stored
- **`train=True`**
 - Instructs PyTorch to load the 60,000 training images
- **`download=True`**
 - Automatically downloads the dataset if it is not already present in the root directory

Dataset Configuration and Loading

To load the datasets, `torchvision.datasets.MNIST` is used with parameters that ensure the training and test sets are compatible.

```
# Load MNIST dataset
train_dataset = torchvision.datasets.MNIST(
    root='./data', train=True, download=True, transform=transform
)
test_dataset = torchvision.datasets.MNIST(
    root='./data', train=False, download=True, transform=transform
)
```

- **download=True**
 - Ensures that if the MNIST data is not already present in the `root` directory, it will be automatically downloaded
- **transform=transform**
 - Applies the previously defined preprocessing steps (like `ToTensor` and `Normalize`) to every image automatically
- **Consistency Requirements**
 - Both the training and test datasets must use the same `transform` and the same `root` storage location to ensure the model is evaluated on data formatted identically to what it was trained on

Creating Data Loaders

`DataLoader` objects are used to wrap the datasets and handle the logistics of feeding data into the model.

```
# Create data loaders
train_loader = DataLoader(train_dataset, batch_size=60, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=1000, shuffle=False)
```

- **train_loader**
 - Uses a `batch_size` of 60

- `shuffle=True` is used to randomize the order of images in every epoch, which helps prevent the model from learning the order of the data
- **`test_loader`**
 - Uses a much larger `batch_size` of 1000
 - `shuffle=False` because the order does not matter during evaluation

Creating Data Loaders

`DataLoader` objects are used to serve batches of data to the model during training and testing.

```
# Create data loaders
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=1000, shuffle=False)
```

Training Loader Configuration

- **`batch_size=64`**
 - Each batch contains 64 images
- **`shuffle=True`**
 - Images are presented in a different random order for every epoch
 - **[Why shuffle?]** Because datasets are often organized by class; shuffling prevents the model from learning the order of the data rather than the actual features

Test Loader Configuration

- **`batch_size=1000`**
 - Uses a much larger batch size than training
 - **[Why larger batches?]** Since we are only testing and not calculating gradients, we don't need the small, frequent updates required for training
- **`shuffle=False`**
 - The order of the test data remains constant

The Importance of Shuffling

- **[Why shuffle in training?]** To ensure each batch contains a variety of data points

- Without shuffling, a model might see long sequences of the same class (e.g., 6,000 zeros in a row)
- This causes the model to learn unintended patterns, such as the position of data in the set, rather than the actual features that define a digit
- **Testing vs. Training**
 - Shuffling is not required for testing
 - During testing, the model has already finished learning and is simply being evaluated on its ability to recognize the inputs

Creating the Neural Network

To build a custom architecture in PyTorch, create a class that inherits from `nn.Module` to gain access to all built-in neural network functionalities.

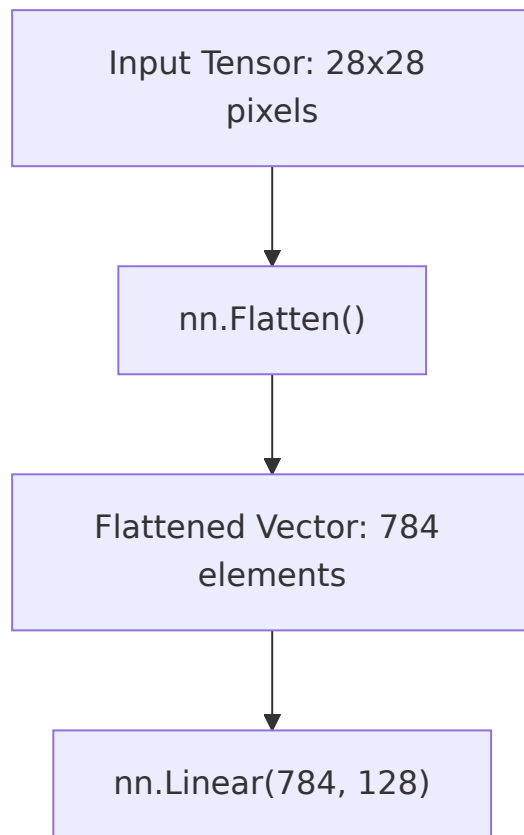
```
class MNISTClassifier(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.flatten = nn.Flatten()  
        self.layers = nn.Sequential(  
            nn.Linear(784, 128),  
            nn.ReLU(),  
            nn.Linear(128, 10)  
        )  
  
    def forward(self, x):  
        x = self.flatten(x)  
        x = self.layers(x)  
        return x
```

The `nn.Flatten` Layer

- **[Why use it?]** MNIST images are loaded as tensors with specific spatial dimensions (e.g., 28x28 pixels).
- Linear layers (like `nn.Linear`) expect a 1D vector of features rather than a 2D grid of pixels.
- `nn.Flatten` transforms the multidimensional image tensor into a single flat vector so it can be processed by the subsequent layers.

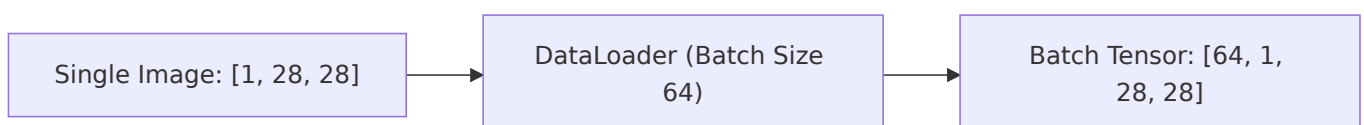
MNIST Image Dimensions

- A single MNIST image is a 28 *times* 28 pixel grid.



MNIST Tensor Dimensions

- **Single Image Structure**
 - A single MNIST image is represented as a [1, 28, 28] tensor
 - The dimensions follow the pattern: [channels, height, width]
 - **Channels:** A value of 1 indicates a grayscale image (a single brightness value per pixel)
 - **Height & Width:** 28 *times* 28 pixels
- **Batch Dimension**
 - When training on batches, PyTorch adds an extra dimension at the beginning to represent the number of samples
 - For a batch size of 64, the tensor shape becomes [64, 1, 28, 28]



The Flattening Process

- **[Why use it?]** Linear layers require a single long row of numbers (a 1D vector) per image rather than a 2D grid of pixels.
- `nn.Flatten` reshapes each 28×28 image into a single row of 784 values ($28 \times 28 = 784$).
- This transformation changes the batch tensor shape from `[64, 1, 28, 28]` to `[64, 784]`.



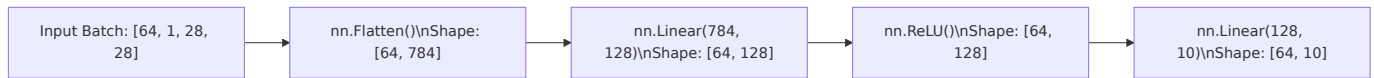
Neural Network Architecture Details

Using `nn.Sequential` allows for stacking layers in a clear, logical order. In the `MNISTClassifier` example, the data undergoes the following transformations:

1. **Flattening:** The input batch of shape `[64, 1, 28, 28]` is converted to `[64, 784]` via `nn.Flatten()`.
2. **Hidden Layer:** `nn.Linear(784, 128)` transforms the 784 pixel values into 128 hidden features.
3. **Activation:** `nn.ReLU()` is applied to the hidden features.
 - **[How it works]** It keeps positive values as they are and zeros out all negative values.
4. **Output Layer:** `nn.Linear(128, 10)` takes the 128 hidden features and maps them to 10 final outputs (representing the 10 possible digit classes).

```
self.layers = nn.Sequential(  
    nn.Linear(784, 128),  
    nn.ReLU(),  
    nn.Linear(128, 10)  
)
```

Summary of Tensor Shape Transformations



Defining the Data Flow with `forward`

- The `forward` method defines how input data moves through the network's layers
- In the `MNISTClassifier`, the flow is:
 1. Receive input `x`
 2. Flatten the input using `self.flatten(x)`
 3. Pass the flattened data through the sequential layers via `self.layers(x)`
 4. Return the final output

```
def forward(self, x):  
    x = self.flatten(x)  
    x = self.layers(x)  
    return x
```

Summary of the Prepared Pipeline

With the architecture defined, two core components are now ready for the training phase:

- **Data Pipeline:** A system that loads and pre-processes MNIST images
- **Neural Network:** A model architecture capable of processing those images

The next step involves bringing the model to life through training, which requires setting up an optimizer and defining a training loop.

Device and Optimizer

Hardware Selection

- Determine whether to use a GPU or CPU for computation
 - If CUDA is available, use the GPU for faster processing
 - Otherwise, fall back to the CPU

```
# Check for GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using {device}')
```

Model and Training Configuration

- **[Model Initialization]** Create the model instance and immediately move it to the selected device
- **[Optimization Setup]** Define how the model's errors are measured and how it learns from them

```
# Initialize model and move to device
model = MNISTClassifier().to(device)

# Loss function and optimizer
loss_function = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

- `nn.CrossEntropyLoss()` : The function used to calculate the difference between predicted and actual labels
- `torch.optim.Adam` : The optimization algorithm used to update the model parameters based on the loss, with a learning rate (`lr`) of 0.001

Device Consistency

- **[Critical Requirement]** Both the model and the training data must be moved to the same device
 - If the model is on the GPU but the data is on the CPU (or vice versa), PyTorch will throw an error during the training process

Loss and Optimization Details

- `nn.CrossEntropyLoss()` : Used for classification tasks
 - It is ideal for picking one class out of many, such as identifying a digit from zero through nine

- `torch.optim.Adam` : An optimizer that adapts its learning rate as training progresses
 - In this setup, it is configured with a learning rate (`lr`) of 0.001

Adam Optimizer Behavior

- Adapts its learning rate based on training stability
 - Makes larger adjustments early on when gradients are noisy
 - Performs smaller corrections later as training stabilizes

Training Function Setup

- **[Function Definition]** A function designed to train the model for one epoch
- **[Inputs]** The function requires five specific arguments:
 - `model` : The neural network to be trained
 - `train_loader` : The data loader providing the batches
 - `loss_function` : The metric used to calculate error
 - `optimizer` : The algorithm used to update parameters
 - `device` : The hardware (CPU/GPU) where computation occurs

```
def train_epoch(model, train_loader, loss_function, optimizer, device):  
    model.train()  
    running_loss = 0.0  
    correct = 0  
    total = 0
```

- `model.train()` : Puts the model into training mode
- **[Tracking Variables]** Variables used to monitor performance during the epoch:
 - `running_loss` : Accumulates the loss values from each batch
 - `correct` : Tracks the number of correct predictions
 - `total` : Tracks the total number of samples processed

Training Loop Implementation

- **[Batch Iteration]** The function loops over all batches using `enumerate(train_loader)`
- **[Device Management]** Within each batch, both the input `data` and the `target` labels must be moved to the specified `device` to ensure compatibility with the model

- `data, target = data.to(device), target.to(device)`

- **[The Training Step Sequence]**

1. `optimizer.zero_grad()` : Clears any leftover gradients from the previous iteration to prevent accumulation
2. `output = model(data)` : Performs the **forward pass** by passing the data through the model to get predictions
3. `loss = loss_function(output, target)` : Computes the error by comparing the model's output to the true labels
4. `loss.backward()` : Performs **backpropagation** to calculate the gradients of the loss with respect to the model parameters
5. `optimizer.step()` : Updates the model's weights based on the calculated gradients

```
for batch_idx, (data, target) in enumerate(train_loader):
    data, target = data.to(device), target.to(device)
    optimizer.zero_grad()
    output = model(data)
    loss = loss_function(output, target)
    loss.backward()
    optimizer.step()
```

Tracking Training Progress

- **[Updating Metrics]** Within the batch loop, track the cumulative loss and accuracy:
 - `running_loss += loss.item()` : Adds the scalar loss value to the running total
 - `_, predicted = output.max(1)` : Retrieves the index of the highest score from the model output, representing the predicted class
 - `total += target.size(0)` : Increments the total count of samples processed
 - `correct += predicted.eq(target).sum().item()` : Compares predictions to true labels and adds the count of matches to the correct total
- **[Periodic Logging]** To avoid flooding the console, print updates every 100 batches:
 - **[Calculation]** Average loss and accuracy are calculated based on the current accumulated values:
 - `avg_loss = running_loss / 100`
 - `accuracy = 100. * correct / total`

- **[Output Format]** The print statement includes the progress through the epoch (calculated as $(\text{batch_idx} * 64) / 60000$) along with the loss and accuracy:

```
# Track progress
running_loss += loss.item()
_, predicted = output.max(1)
total += target.size(0)
correct += predicted.eq(target).sum().item()

# Print every 100 batches
if batch_idx % 100 == 0 and batch_idx > 0:
    avg_loss = running_loss / 100
    accuracy = 100. * correct / total
    print(f'[{batch_idx * 64}/{60000}] '
          f'Loss: {avg_loss:.3f} | Accuracy: {accuracy:.1f}%')
```

- **[Observing Convergence]** Monitoring the output allows you to see if the model is learning:
 - A successful training run should show the loss value decreasing over time (e.g., dropping from 0.641 to 0.168) and accuracy increasing.

Epoch	Loss	Accuracy
1	0.641	81.58%
1	0.321	90.25%
1	0.284	91.84%
1	0.235	93.23%
1	0.200	93.94%
1	0.182	94.52%
1	0.180	94.31%
1	0.175	95.00%

1	0.168	94.91%
---	-------	--------

Evaluation Phase

- **[Purpose]** To test the model's performance on new, unseen data (the test set) rather than the data it has already seen during training
- **[Key Differences from Training]**
 - `model.eval()` : Switches the model into evaluation mode
 - `with torch.no_grad()` : Wraps the evaluation logic to disable gradient calculations
 - **[Why?]** Since we are not performing backpropagation during evaluation, calculating gradients is unnecessary and computationally expensive

```
def evaluate(model, test_loader, device):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for inputs, targets in test_loader:
            inputs, targets = inputs.to(device), targets.to(device)
            outputs = model(inputs)
            _, predicted = outputs.max(1)
            total += targets.size(0)
            correct += predicted.eq(targets).sum().item()

    return 100. * correct / total
```

Evaluation Logic Details

- **[Efficiency]** Using `with torch.no_grad()` saves memory and increases speed
 - **[Why?]** Because we are only performing inference, not backpropagation
- **[Process]** The evaluation is a simple measurement of accuracy:
 1. Run each batch through the model
 2. Use `outputs.max(1)` to identify the predicted class
 3. Compare predictions to true labels to count correct matches

4. Return the final accuracy percentage

- **[Key Omissions]** Unlike the training loop, the evaluation phase does NOT include:
 - Optimizer steps
 - Loss tracking for backpropagation
 - Weight updates

Putting It All Together

- The complete workflow involves running the training function for a specified number of epochs (e.g., 10 full passes through the dataset)

```
# Training loop
num_epochs = 10
for epoch in range(num_epochs):
    print(f'\nEpoch: {epoch+1}')
    train_epoch(model, train_loader, loss_function, optimizer, device)
    accuracy = evaluate(model, test_loader, device)
    print(f'Test Accuracy: {accuracy:.2f}%')
```

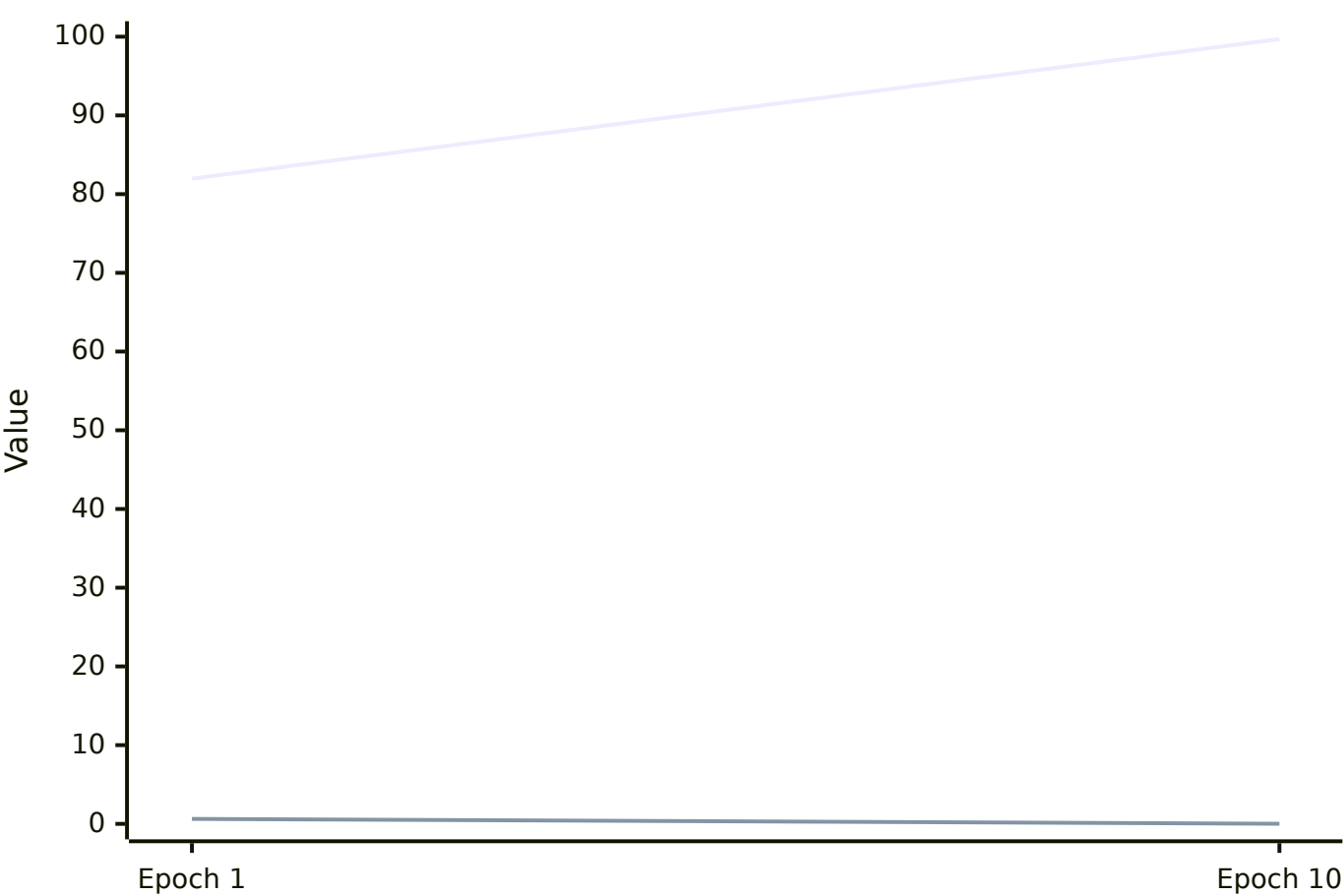
The Purpose of Multiple Epochs

- Iterating through the training dataset multiple times (epochs) allows the model to refine its understanding of specific features
 - For example, it learns the subtle nuances that distinguish a "2" from a "7"
- **[Generalization vs. Memorization]** Evaluating on the test set after each epoch is crucial
 - It helps determine if the model is learning generalizable patterns
 - Or if it is simply memorizing the training data (overfitting)

Training Progression

- As training progresses through more epochs, the model typically shows a decrease in loss and an increase in accuracy

Training Progress (Epoch 1 vs Epoch 10)



Metric	Epoch 1 (Start)	Epoch 10 (End)
Loss	0.627	0.013
Accuracy	81.97%	99.69%

Monitoring Training Convergence

- **[When to stop]** When accuracy stops improving, it is often a sign that the model has finished learning for the current setup
 - **[Why?]** Continuing to train beyond this point may not yield significant gains and could potentially lead to overfitting

Epoch	Loss Trend	Accuracy Trend
Epoch 1	High (e.g., 0.627)	Lower (e.g., 81.97%)
Epoch 2	Decreasing (e.g., 0.115)	Increasing (e.g., 96.78%)
Epoch 10	Very Low (e.g., 0.013)	Very High (e.g., 99.69%)

- **[Goal]** The ideal outcome is a very tiny loss and a very high accuracy percentage